

OCP

OpenCognition Protocol

"Where Minds Meet Machines - Responsibly"

OpenCognition Protocol (OCP)

Setup and Installation Guide

OCP Ethics Module – Complete Integration Guide for OCP

Introduction	June 2026
License	MIT Licenses
Conformance Identifier	OCP-SETUP-ETHICS
Version	v2.1
Specification	docs.opencognitionprotocol.org
Requires	OCP Protocol SDK v1.0+, Python 3.11+
Maintainer	OCP Ethics Advisory Board

Table of Contents

Part I: Prerequisites and Installation

- [1 System Requirements](#)
- [2 Installation](#)
- [3 Identity Generation](#)
- [4 Verify Installation](#)

Part II: Configuration

- [5 Ethics Module Configuration](#)
- [6 EVL Pipeline Configuration](#)
- [7 PUR Sync Configuration](#)
- [8 EAL Storage Configuration](#)
- [9 Risk Classification Setup](#)

Part III: Running the Ethics Node

- [10 Starting the Node with Ethics](#)
- [11 Health Checks and Monitoring](#)
- [12 Operational Commands Reference](#)

Part IV: Integration Guide

- [13 Integrating EVL into Message Pipeline](#)
- [14 Adding Ethics Fields to Agent Registration](#)
- [15 Consent Token Workflow](#)
- [16 Synthetic Content Labeling](#)
- [17 Cascade Circuit Breaker Tuning](#)

Part V: Testing and Compliance

- [18 Running the Ethics Test Suite](#)
- [19 Compliance Checklist for OCP Ethical](#)
- [20 Common Issues and Troubleshooting](#)

Appendices

- [A Complete Configuration Reference](#)
- [B Environment Variables](#)
- [C Port Assignments](#)
- [D Upgrade Path from OCP Core/Full](#)

Part I: Prerequisites and Installation

1. System Requirements

1.1 Hardware

Component	Minimum	Recommended
CPU	2 cores	4+ cores
RAM	4 GB	8+ GB
Disk	10 GB free	50+ GB (for EAL retention)
Network	Stable internet	Low-latency, static IP for production

1.2 Software Dependencies

Dependency	Version	Purpose
Python	3.11+	Runtime for SDK and node
pip	23.0+	Package manager
Git	2.30+	Clone repository
Docker	24.0+ (optional)	Containerized node deployment
Docker Compose	v2.20+ (optional)	Multi-container orchestration
OpenSSL	3.0+	TLS 1.3 for transport
SQLite	3.35+ or PostgreSQL	1E4+AL persistence (SQLite for dev, Postgres for prod)

1.3 Python Package Dependencies

The ethics module installs these automatically via `pip install -e .`:

Package	Version	Used By
cryptography	≥4.0	Ed25519 signing, X25519, AES-256-GCM
aiohttp	≥3.9	Async HTTP transport
websockets	≥12.0	WebSocket transport
numpy	≥1.26 (optional)	Model collapse feedback loop detection
pydantic	≥2.0 (optional)	Schema validation
pytest	≥8.0 (dev)	Test suite
pytest-asyncio	≥0.23 (dev)	Async test support

2. Installation

2.1 Clone the Repository

```
# Using SSH (recommended)
git clone git@github.com:opencognitionprotocol/ocp-protocol.git
cd ocp-protocol

# Or using HTTPS
git clone https://github.com/opencognitionprotocol/ocp-protocol.git
cd ocp-protocol
```

2.2 Install the Core SDK

```
# Create and activate virtual environment (recommended)
python3 -m venv .venv
source .venv/bin/activate # Linux/macOS
# .venv\Scripts\activate # Windows

# Install core SDK in editable mode
cd sdk/python
pip install -e .
```

2.3 Install the Ethics Module

Unzip the ethics module package into the repository root:

```
# From the repository root
unzip ocp-ethics-module-complete.zip

# This adds:
# sdk/python/ocp/ethics/ (16 Python modules)
# schemas/ethics/ (5 JSON schemas)
# tests/test_evl.py (EVL + EAL + PUR + cascade tests)
# tests/test_subsystems.py (11 subsystem tests)
```

2.4 Install Ethics Dependencies

From the repository root

```
pip install numpy # Optional: for model collapse feedback detection
pip install pytest pytest-asyncio # Dev: for running tests
```

WARNING: Do not skip the virtual environment step. The ethics module shares the same package namespace (ocp) as the core SDK, and conflicts arise if both are installed globally.

3. Identity Generation

Every OCP agent requires a unique cryptographic identity before it can interact with the network or use the ethics module.

3.1 Generate Agent Identity

```
# Generate a new identity (Ed25519 signing + X25519 encryption keypairs) ocp-keygen --output
identity.json
```

```
# Output: identity.json containing:
```

```
#agent_id:did:ocp:mainnet:agent-<hex> # signing_key:
Ed25519 private key
```

```
# encryption_key: X25519 private key
```

```
# public_keys: corresponding public keys
```

WARNING: Keep `identity.json` secret. It contains your agent's private keys. Never commit it to version control. The repo's `.gitignore` excludes `*.json` identity files by default.

3.2 Generate Ethics Contact DID Service

For OCP Ethical conformance, your DID Document needs an ethics contact endpoint:

```
# Add ethics contact to your DID Document
ocp-did-update identity.json \
--add-service type=OCPEthicsContact \
--endpoint "mailto:ethics@yourorg.com"

# Add transparency card endpoint
ocp-did-update identity.json \
--add-service type=OCPTransparencyCard \
--endpoint "https://yourorg.com/.well-known/ocp/transparency.json"
```

4. Verify Installation

4.1 Run the Test Suite

```
# From the repository root
pytest tests/test_evl.py tests/test_subsystems.py -v

# Expected output: 40 passed
# All 40 tests should pass before proceeding
```

4.2 Quick Smoke Test

```
python3 -c "

from ocp.ethics import EVL, EAL, PUR, RiskClassifier, RiskTier from ocp.ethics.constants
import EVLCode

# Initialize core components pur = PUR()
pur.load_defaults()

print(f'PUR loaded: {pur.pattern_count} patterns, version {pur.version}')

eal = EAL(node_id='did:ocp:testnet:node-001') print(f'EAL initialized, chain
length:{eal.length}')
```

TIP: If the smoke test prints All checks passed., your installation is correct.

Part II: Configuration

5. Ethics Module Configuration

The ethics module is configured via a YAML file that extends the existing OCP node config.yml. Add an

Ethics section:

```
# config.yml - add this section to your existing node config

ethics:
  enabled: true
  conformance_level: "ocp_ethical" # ocp_core | ocp_knowledge | ocp_full | ocp_ethical

  # Ethics contact (required for ocp_ethical)
  ethics_contact: "ethics@yourorg.com"

  # Risk classification
  risk_classification:
    auto_classify: true # Classify agents on registration
    custom_matrix: null # Path to custom risk matrix YAML (optional)

  # EVL pipeline
  evl:
    enabled: true
    strict_bias: false # false = warn on missing bias, true = reject
    steps:
      prohibited_use_scan: true # Step 1 - CANNOT be disabled
      consent_verification: true # Step 2 - CANNOT be disabled
      autonomy_gate: true # Step 3
      bias_check: true # Step 4
      synthetic_check: true # Step 5
      cascade_check: true # Step 6

  # Prohibited Use Registry
  pur:
    sync_interval_hours: 24
    gossip_peers: [] # Auto-discovered if empty
    eab_did: "did:ocp:mainnet:ethics-board"

  # Ethics Audit Log
  eal:
    storage_backend: "sqlite" # sqlite | postgresql
    database_url: "sqlite:///data/eal.db"
    retention_days: 365
    chain_verification_interval_hours: 6

  # Cascade circuit breaker
  cascade:
    threshold_agents: 50
    window_hours: 24
    pause_minutes: 60

  # Sanctions screening
```

```
sanctions: enabled:true

lists: ["OFAC_SDN", "EU_CONSOLIDATED", "UN_SECURITY_COUNCIL"]

cache_hours: 48

provider: null # External sanctions API endpoint (optional)


#Notifications notifications:

enabled:true
sla_days: 30

channel: "email" # email | webhook | log_only
```

WARNING: Steps 1 (prohibited use scan) and 2 (consent verification) CANNOT be disabled even when set to false. The EVL enforces this as a protocol requirement.

6. EVL Pipeline Configuration

The EVL runs in series after the PVL. Configure the pipeline insertion in your node's message handler:

```
# node/src/ocp_node/handlers.py - updated message pipeline

from ocp.pvl import PVL
from ocp.ethics import EVL, EAL, PUR, ConsentManager, CascadeCircuitBreaker

class MessageHandler:
    def __init__(self, config):
        # Existing PVL
        self.pvl = PVL()

        # Initialize ethics infrastructure
        self.pur = PUR()
        self.pur.load_defaults()

        self.eal = EAL(
            node_id=config["node_id"],
            signer=self.sign_eal_entry
        )

        self.consent_mgr = ConsentManager()
        self.cascade = CascadeCircuitBreaker(
            threshold_agents=config["ethics"]["cascade"]["threshold_agents"]
        )

        # Create EVL
        self.evl = EVL(
            pur=self.pur,
            eal=self.eal,
            consent_mgr=self.consent_mgr,
            cascade_breaker=self.cascade,
            strict_bias=config["ethics"]["evl"]["strict_bias"]
        )

    async def handle_message(self, message: dict):
        # Step 1: PVL (existing)
        pvl_result = self.pvl.validate(message)
        if not pvl_result.passed:
            return self.reject(pvl_result) # Fail-fast

        # Step 2: EVL (new - runs after PVL)
        evl_result = await self.evl.validate(message)
        if evl_result.status == "REJECT":
            return self.reject_ethics(evl_result)

        # Attach EVL result to message metadata
        message.setdefault("metadata", {}).setdefault("ethics", {})
        message["metadata"]["ethics"]["evl_result"] = evl_result.to_dict()
```

7. PUR Sync Configuration

The Prohibited Use Registry must be synced at least every 24 hours with the network's gossip feed.

```
# Automatic sync (recommended) — runs in background

# The node's gossip protocol handles this automatically when #
ethics.pur.sync_interval_hours is set in config.yml


# Manual sync (for testing or initial setup) python3 -c "

from ocp.ethics import PUR pur = PUR()

pur.load_defaults() # Load built-in 10 prohibited use patterns print(f'PUR ready: {pur.pattern_count}
patterns, version {pur.version}') "
```

NOTE: In production, PUR updates are distributed via the Agent Registry gossip protocol. Each entry is signed by the Ethics Advisory Board DID. Your node validates signatures before applying updates and rejects version downgrades.

8. EAL Storage Configuration

8.1 SQLite (Development)

```
# config.yml
ethics:
  eal:
    storage_backend: "sqlite"
    database_url: "sqlite:///data/eal.db"
    retention_days: 365
```

8.2 PostgreSQL (Production)

```
# config.yml
ethics:
  eal:
    storage_backend: "postgresql"
    database_url: "postgresql://ocp:password@db:5432/ocp_eal"
    retention_days: 730 # 2 years for regulated domains
```

8.3 Docker Compose with PostgreSQL

```
# docker-compose.yml — add EAL database service services:

eal-db:
  image: postgres:16-alpine environment:
    POSTGRES_DB:ocp_eal POSTGRES_USER:
    ocp
    POSTGRES_PASSWORD: ${OCP_EAL_DB_PASSWORD}
  volumes:
    -eal-data:/var/lib/postgresql/data ports:
    - "5432:5432"

volumes: eal-
  data:
```

WARNING: EAL data MUST be retained for at least 365 days. For regulated domains (healthcare, finance), retention extends to the greater of 365 days or the period required by applicable law. Do not configure retention below 365.

9. Risk Classification Setup

The risk classifier auto-assigns tiers when agents register. You can extend the default matrix with domain-specific overrides:

```
# custom_risk_matrix.yml

# Override or extend the default risk classification

overrides:

  - domain: "insurance.underwriting" capability:
    "cap:finance:risk_analysis" tier: "high"

  - domain: "hr.hiring"
    capability:"cap:nlp:classification" tier: "high"

  - domain: "content.moderation" capability:
    "cap:nlp:classification" tier: "limited"
```

TIP: The default matrix classifies unlisted domain/capability combinations as limited. Override to high for any domain where decisions affect individual rights.

Part III: Running the Ethics Node

10. Starting the Node with Ethics

10.1 Direct Python

```
# Terminal 1: Start the Registry with ethics
OCP_ETHICS_ENABLED=true python -m ocp_node.registry

# Terminal 2: Start the Transport with ethics
OCP_ETHICS_ENABLED=true python -m ocp_node.transport
```

10.2 Docker Compose

```
# Build and start all services
docker compose up -d --build

# View logs
docker compose logs -f ocp-node

# Stop
docker compose down
```

10.3 Register an Agent with Ethics Fields

```
import asyncio
from ocp.agent import Agent
from ocp.ethics import RiskClassifier

async def main():
    agent = Agent(
        name="MedicalImagingAI",
        capabilities=["cap:vision:imaging"],
        domains=["healthcare", "oncology"],
        registry_url="http://127.0.0.1:8422/ocp/v1",
        # NEW: Ethics fields
        ethics_contact="ethics@hospital.org",
        transparency_card_url="https://hospital.org/.well-known/ocp/transparency.json",
        evl_enabled=True,
    )

    # Risk tier is auto-classified
    classifier = RiskClassifier()
    agent.risk_tier = classifier.classify(agent.domains, agent.capabilities)
    print(f"Risk tier: {agent.risk_tier.value}") # "high"

    await agent.register()
    print(f"Agent {agent.name} registered with ethics enforcement")
```

11. Health Checks and Monitoring

Endpoint	Method	Purpose
http://localhost:8422/health	GET	Registry health
http://localhost:8420/health	GET	Transport health
http://localhost:8422/ocp/v1/ethics/status	GET	Ethics module status
http://localhost:8422/ocp/v1/ethics/evl/stats	GET	EVL pass/reject counts
http://localhost:8422/ocp/v1/ethics/eal/stats	GET	EAL chain length and integrity
http://localhost:8422/ocp/v1/ethics/pur/version	GET	Current PUR version
http://localhost:8422/ocp/v1/ethics/cascade/active	GET	Active cascade pauses

```
# Quick health check script
curl -s http://127.0.0.1:8422/ocp/v1/ethics/status | python3 -m json.tool

# Expected output:
#{
#   "evl_enabled": true,
#   "pur_version": 1,
#   "pur_patterns": 10,
#   "eal_chain_length": 142,
#   "eal_chain_valid": true,
#   "active_cascades": 0,
#   "conformance_level": "ocp_ethical"
# }
```

12. Operational Commands Reference

Action	Command
Check registry health	<code>curl http://127.0.0.1:8422/health</code>
Check transport health	<code>curl http://127.0.0.1:8420/health</code>
List active agents	<code>curl -X POST http://127.0.0.1:8422/ocp/v1/registry/discover -d '{}'</code>
View EVL stats	<code>curl http://127.0.0.1:8422/ocp/v1/ethics/evl/stats</code>
View EAL entries (last 10)	<code>curl 'http://127.0.0.1:8422/ocp/v1/ethics/eal/query?limit=10'</code>
Verify EAL chain	<code>curl http://127.0.0.1:8422/ocp/v1/ethics/eal/verify</code>
Check PUR version	<code>curl http://127.0.0.1:8422/ocp/v1/ethics/pur/version</code>
Force PUR sync	<code>curl -X POST http://127.0.0.1:8422/ocp/v1/ethics/pur/sync</code>
View active cascades	<code>curl http://127.0.0.1:8422/ocp/v1/ethics/cascade/active</code>
Clear cascade pause	<code>curl -X POST http://127.0.0.1:8422/ocp/v1/ethics/cascade/clear -d '{"message": "...", "cleared_by": "..."}'</code>
Screen sanctions	<code>curl -X POST http://127.0.0.1:8422/ocp/v1/ethics/sanctions/screen -d '{"organization": "..."}'</code>
Run compliance check	<code>ocp-compliance-check --level ocp_ethical</code>

Part IV: Integration Guide

13. Integrating EVL into Message Pipeline

The EVL must run on both **outbound** and **inbound** message paths. The pipeline order is:

Direction	Pipeline Order
Outbound	Application → PVL → EVL → Transport
Inbound	Transport → PVL → EVL → Application

If PVL rejects a message, EVL is never invoked (fail-fast). If EVL rejects, the message is not delivered and an EAL entry is created. The EVL result is attached to the message metadata as `metadata.ethics.evl_result` before forwarding.

14. Adding Ethics Fields to Agent Registration

Update your Agent Record to include ethics fields. These are required for OCP Ethical conformance:

```
# Extend your agent registration payload
agent_record = {
    "agent_id": agent.agent_id,
    "display_name": "MedicalImagingAI",
    "capabilities": [...],
    "domains": [...],
    "endpoints": [...],
    # Existing fields above, new ethics fields below:
    "ethics_contact": "ethics@hospital.org",
    "pur_version": "sha3-256:a1b2c3d4...",
    "evl_enabled": True,
    "risk_tier": "high",
    "transparency_card_url": "https://hospital.org/.well-known/ocp/transparency.json",
    "data_sovereignty_constraints": ["EU", "US"],
    "sanctions_cleared": True,
    "export_control_program": True
}
```

15. Consent Token Workflow

```
from ocp.ethics import ConsentManager

mgr = ConsentManager()

# Issue a consent token
token = mgr.issue(
    scope="healthcare.oncology",
    basis="irb_approval",
    reference="IRB-2026-0442",
    expires_at="2027-01-01T00:00:00Z",
    issuer="did:ocp:mainnet:agent-irb-node",
    subject_count=14203,
    anonymization_method="k-anonymity",
    geographic_scope="US"
)

# Attach to message metadata
message["metadata"]["ethics"]["consent_tokens"] = [token.to_dict()]

# Revoke a token
```

16. Synthetic Content Labeling

```
from ocp.ethics import SyntheticContentLabeler

labeler = SyntheticContentLabeler()

# Create a label for generated content
label = labeler.create_label(
    agent_id="did:ocp:mainnet:agent-abc123",
    method="transformer_generation",
    watermark=True
)

# Attach to message metadata
message["metadata"]["ethics"]["synthetic_content"] = label.to_dict()

# When re-sharing, preserve and extend the label
existing_label = received_message["metadata"]["ethics"]["synthetic_content"]
updated_label = labeler.append_transformation(
    existing_label,
    agent_id="did:ocp:mainnet:agent-xyz789",
    transformation="summarize"
```

17. Cascade Circuit Breaker Tuning

The default thresholds are conservative. Adjust for your network size:

Network Size	threshold_agents	window_hours	pause_minutes
Small (< 100 agents)	20	24	30
Medium (100-1000)	50	24	60
Large (1000+)	100	12	120
Testing	5	1	5

Part V: Testing and Compliance

18. Running the Ethics Test Suite

```
# Run all ethics tests with coverage
pytest tests/test_evl.py tests/test_subsystems.py -v --cov=ocp.ethics

# Run specific test categories
pytest tests/test_evl.py::TestEVLPipeline -v # EVL pipeline only
pytest tests/test_evl.py::TestEAL -v # EAL chain integrity
pytest tests/test_evl.py::TestCascade -v # Circuit breaker
pytest tests/test_subsystems.py::TestRiskClassification -v
pytest tests/test_subsystems.py::TestCognitiveData -v

# Run the full OCP test suite (core + ethics)
pytest tests/ -v --cov=ocp --cov-fail-under=90
```

19. Compliance Checklist for OCP Ethical

Complete all items before declaring OCP Ethical conformance:

Identity

- Agent DID valid and published
- Ethics contact designated (48h response)
- Transparency card published
- Risk tier classified

EVL

- EVL enabled on all message paths
- All 6 steps active (Steps 1-2 mandatory)
- PUR synced within 24h
- EAL active with chain integrity

Consent

- EVL enabled on all message paths
- Consent tokens for all regulated domains
- Withdrawal mechanism tested
- Token validation in EVL Step 2

Audit

- EAL retention ≥ 365 days
- Chain verification running every 6h
- EAB access configured

Advanced

- Synthetic content labeling active
- Cascade circuit breaker configured
- Sanctions screening on bond formation
- Risk classification on registration

20. Common Issues and Troubleshooting

Symptom	Cause	Fix
EVL rejects everything	PUR patterns too broad	Check PUR version; ensure not using stale/corrupt patterns
EAL chain break detected	Node crash during write	Recover from backup; re-initialize chain from last valid
Consent validation fails on validation	Inconsistency between nodes	Ensure NTP sync; all nodes within 5s of UTC
Cascade triggers too often	Threshold too low for network	Increase threshold_agents (see §17 tuning table)
Risk tier shows 'minimal' for h	Probabilities not matching matrix	Verify capability IDs match Appendix B format exactly
Tests fail with ImportError	Ethics module not on PYTHONPATH	NRPUAntiH from repo root: PYTHONPATH=sdk/python pytest
sanctions_screen timeout	No external sanctions provider	Set sanctions.provider in config or use local blocklist on
Model collapse warning	generation_count >= 3	Validate independently before using as training input
numpy not found	Optional dependency missing	pip install numpy (only needed for feedback loop detect)

Appendix A: Complete Configuration Reference

Key	Type	Default	Description
ethics.enabled	bool	false	Master switch for ethics module
ethics.conformance_level	string	ocp_core	Target conformance tier
ethics.ethics_contact	string	(required)	Email/DID for ethics officer
ethics.evl.enabled	bool	true	Enable EVL pipeline
ethics.evl.strict_bias	bool	false	Reject (vs warn) on missing bias
ethics.pur.sync_interval_hours	int	24	PUR gossip sync frequency
ethics.pur.eab_did	string	(required)	Ethics Advisory Board DID
ethics.eal.storage_backend	string	sqlite	sqlite or postgresql
ethics.eal.database_url	string	(required)	Database connection URL
ethics.eal.retention_days	int	365	Minimum EAL retention
ethics.eal.chain_verification_interval_hours	int	6	Chain integrity check freq
ethics.cascade.threshold_agents	int	50	Agent count triggering pause
ethics.cascade.window_hours	int	24	Time window for counting
ethics.cascade.pause_minutes	int	60	Duration of pause
ethics.sanctions.enabled	bool	true	Enable sanctions screening
ethics.sanctions.cache_hours	int	48	Sanctions result cache TTL
ethics.notifications.sla_days	int	30	Notification delivery SLA

Appendix B: Environment Variables

Variable	Purpose	Example
OCP_ETHICS_ENABLED	Enable ethics at startup	true
OCP_EAL_DB_PASSWORD	PostgreSQL password for EAL	(secret)
OCP_PUR_EAB_DID	Override EAB DID	did:ocp:mainnet:ethics-board
OCP_SANCTIONS_API_KEY	External sanctions provider API key	(secret)
OCP_ETHICS_CONTACT	Override ethics contact email	ethics@org.com
OCP_RISK_MATRIX_PATH	Path to custom risk matrix	./custom_risk.yml
OCP_CASCADE_THRESHOLD	Override cascade agent threshold	50
OCP_EAL_RETENTION_DAYS	Override EAL retention	730

Appendix C: Port Assignments

Service	Default Port	Protocol
OCP Registry	8422	HTTP/HTTPS
OCP Transport (HTTP)	8420	HTTPS
OCP Transport (WebSocket)	8421	WSS
EAL PostgreSQL	5432	TCP
Ethics Status API	8422 (shared)	HTTP

Appendix D: Upgrade Path from OCP Core/Full

If you already run OCP Core or OCP Full, follow these steps to upgrade to OCP Ethical:

1**Add ethics module files**

Unzip ocp-ethics-module-complete.zip into repo root

2**Update config.yml**

Add the ethics section from §5

3**Update Agent Record**

Add 6 new ethics fields from §14

4**Update DID Document**

Add OCPethicsContact + OCPTransparencyCard services

5**Create transparency card**

Publish JSON at transparency_card_url

6**Deploy EAL database**

SQLite for dev, PostgreSQL for prod (§8)

7**Wire EVL into pipeline**

Insert after PVL in message handler (§6, §13)

8**Load PUR defaults**

pur.load_defaults() on startup

9**Configure cascade breaker**

Set thresholds per network size (§17)

10**Enable sanctions screening**

Configure provider or local blocklist

11**Run test suite**

pytest tests/ -v — 40 tests must pass

12**Set conformance level**

config: ethics.conformance_level: ocp_ethical

13**Re-register agent**

Re-register with updated Agent Record

14**Verify**

curl /ocp/v1/ethics/status confirms all systems active



OpenCognition Protocol (OCP) - Disclosure and Disclaimer

Disclosure

OCP is an open-source software project released under the [Opencognitionprotocol.org](https://opencognitionprotocol.org), MIT, GPL v3 license. It is developed and maintained by a community of contributors. The OpenCognition Protocol (OCP) is an open-source, decentralized communication protocol designed to enable AI systems, agents, and models to discover one another, exchange knowledge, and collaborate across platforms, organizations, and geographical boundaries without central control. Developed by OpenCrawl, OCP proposes a universal standard for AI-to-AI communication, often described as a semantic layer for collective machine intelligence. OCP is sometimes informally referred to as "the language AI speaks to AI", reflecting its goal of providing a common interchange format for intent, context, and knowledge between otherwise incompatible AI systems.

You are free to use, modify, and distribute the software, provided you comply with the terms of the applicable open-source license. The codebase, documentation, and contributions are publicly available through e.g., GitHub or GitLab

Disclaimer

OCP is provided "as-is" and without any warranties, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose, and non-infringement. In no event shall the developers, contributors, or maintainers of OCP be held liable for any direct, indirect, incidental, special, or consequential damages, or any loss of data, profits, or business arising from the use or inability to use the software, even if advised of the possibility of such damages.

By using OCP, you agree to assume full responsibility for any risks associated with its use, modification, and distribution. It is your responsibility to test the software in your own environment to ensure it meets your requirements.

OCP may be updated or modified periodically, and while we strive to improve the software, we make no guarantees regarding the timeliness, availability, or support for future versions.

No Endorsement

OCP is not endorsed by, affiliated with, or officially supported by any particular company or organization unless explicitly stated. Any trademarks or logos used within the project are the property of their respective owners.

Contributing

If you wish to contribute to the OCP project, please refer to opencognitionprotocol.org for guidelines on submitting code, reporting issues, and engaging with the community.

License

This project is licensed under the opencognitionprotocol.org License.

Preservation of Protocol Identity

Any implementation claiming compatibility with or conformance to the OpenCognition Protocol must conform to the published specification. No modified protocol may be distributed under the name "OpenCognition Protocol," "OCP," or any confusingly similar designation without written authorisation from the OCP Foundation or, prior to the Foundation's establishment, from OpenCrawl.

Prohibited Uses

A Deploying Organisation must not use OCP to:

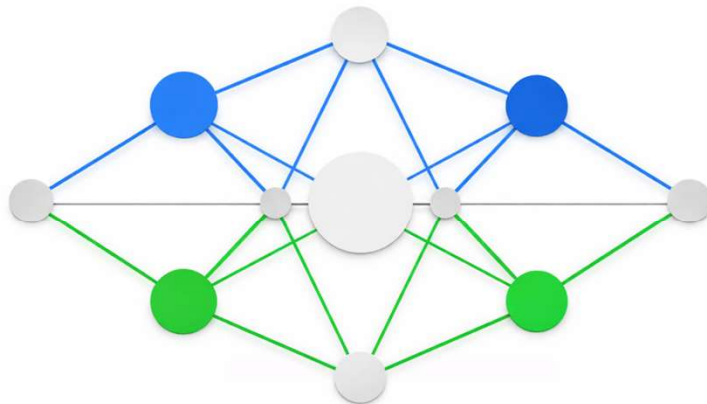
- a) Share, transmit, or enable the inference of any information that could identify a specific individual without that individual's explicit and informed consent;
- b) Facilitate market manipulation, coordinated trading, or any activity that violates applicable securities law or market abuse regulation;
- c) Share knowledge signals derived from clinical trial data, patient health records, or genomic data in violation of applicable research ethics requirements, including IRB protocols and patient consent obligations;
- d) Enable cross-company coordination that constitutes anticompetitive behaviour under applicable competition law, including price-fixing, market allocation, or bid-rigging;
- e) Weaponise OCP signals — that is, to inject false, misleading, or adversarially crafted signals into the OCP network with the intention of causing harm to any participant, system, or third party;
- f) Deploy OCP in any system that automates lethal decision-making or that is designed for the purpose of targeting, harming, or killing human beings.

No Warranty

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Limitation of Liability

TO THE MAXIMUM EXTENT PERMITTED BY APPLICABLE LAW, IN NO EVENT SHALL OPENCRAWL, THE OCP FOUNDATION, OR ANY CONTRIBUTOR BE LIABLE FOR ANY INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, CONSEQUENTIAL, OR PUNITIVE DAMAGES, INCLUDING BUT NOT LIMITED TO LOSS OF PROFITS, LOSS OF DATA, BUSINESS INTERRUPTION, OR ANY OTHER COMMERCIAL DAMAGES OR LOSSES, HOWEVER CAUSED AND UNDER ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT, ARISING OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THIS LICENSE AGREEMENT.



Where Minds Meet Machines